SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CONCEPT MAPPING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO1 – Precision (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 2 – Concept Mapping
Weightage:	20% of CIA	**Total Marks:**	100 (2 Questions × 50 Marks)
CO1 Statement:	*Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)*		
Student Name:	J KARTHIK	**Reg. No.:**	192472136
Section / Batch:	CSE AI / 2024	**Date:**	

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Draw a clear and neat concept map for each question.
- Identify all key concepts and arrange them in a logical hierarchy.
- Show meaningful linkages between concepts using arrows and connecting words.
- Compare related concepts wherever required and justify the relationships clearly.
- Ensure the concept map is well-organized, readable, and properly labelled.

Question Type	Focus Area	Questions	Marks
Concept Mapping	Map algorithm efficiency concepts using asymptotic analysis, search techniques, recurrence relations, and recursion tree method	Q1 – Q2	Q1 –50 Q2 –50
	GRAND TOTAL:	**100 Marks**	

SECTION A – CONCEPT MAPPING

Code of Conduct

*I **J KARTHIK / 192472136** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: J KARTHIK

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

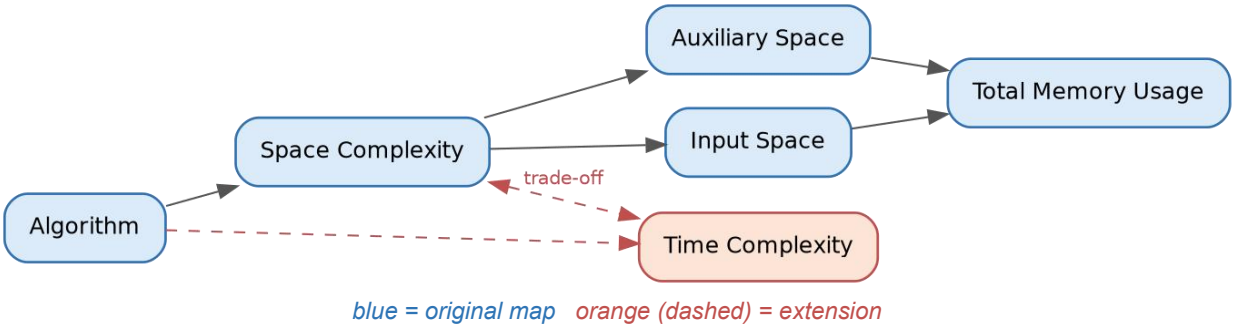
Marks Evaluation:

Question No.	Concept Identification (25%)	Linkages (25%)	Organization (20%)	Integration of Literature (20%)	Clarity (10%)	Total Marks (Each – 50)
Q1						
Q2						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Concept Maps — Data Structures & Algorithms

Q81. Memory Usage: Space Complexity Concept Map



Explain — How total memory usage is calculated

Total memory usage is the sum of two components: Input Space, the memory needed to store the input data itself (e.g., an array of n elements), and Auxiliary Space, the extra memory the algorithm uses beyond the input while it runs (e.g., temporary variables, recursion stack frames, or helper arrays).

Total Memory Usage = Input Space + Auxiliary Space

Space Complexity, as commonly reported (e.g., "O(n)" or "O(1)"), most often refers to Auxiliary Space alone, since Input Space is usually unavoidable and identical across algorithms solving the same problem. The concept map shows this flow: Space Complexity branches into Auxiliary Space and Input Space, which both feed into Total Memory Usage.

Extend — Linking Time Complexity (trade-off)

The map is extended by adding a Time Complexity node connected back to Space Complexity through a dashed "trade-off" edge. This captures the well-known space–time trade-off: an algorithm can often be made faster by using more memory (e.g., memoization, hash tables, precomputed lookup tables), or made more memory-efficient by accepting more computation time (e.g., recomputing values instead of storing them).

Justify — When is higher space usage acceptable?

Higher space usage is justified when:

• The system has abundant memory but a strict time budget (e.g., real-time systems, high-frequency trading, interactive applications where latency matters more than RAM cost).

• The same computation is repeated many times, so caching/memoization amortizes the extra space cost across many calls (e.g., dynamic programming).

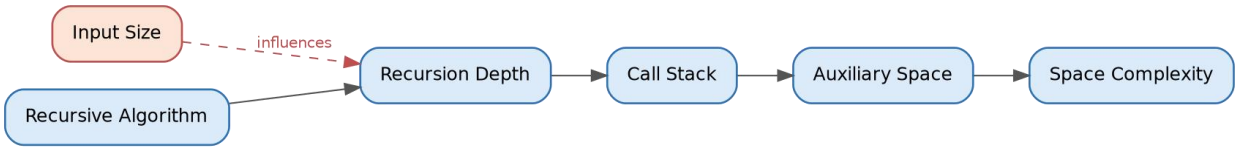
• Memory is cheap relative to the cost of recomputation (e.g., modern servers with large RAM vs. CPU-bound operations).

• Correctness or simplicity benefits from extra space (e.g., storing intermediate results to avoid complex in-place logic that is error-prone).

Interpret — How this map supports memory-efficient design

The map gives designers a structured way to reason about where memory is actually spent, rather than treating "space complexity" as a single opaque number. By separating Input Space from Auxiliary Space, a designer can identify which part is reducible (usually the auxiliary part) and target optimizations there — e.g., converting recursive solutions to iterative ones, using in-place algorithms, or releasing temporary structures early. The linked Time Complexity node further reminds designers that memory savings should be evaluated against acceptable time costs, guiding balanced, context-aware algorithm choices rather than blindly minimizing one metric.

Q82. Recursion Depth and Memory Concept Map



blue = original map orange (dashed) = extension

Explain — How recursion depth affects memory consumption

Each recursive call adds a new frame to the Call Stack, holding that call's local variables, parameters, and return address. Recursion Depth is the maximum number of such nested calls active at once before the recursion unwinds. Since each frame consumes a roughly constant amount of memory, the total Auxiliary Space used is proportional to the recursion depth: more depth means more simultaneously active frames, which directly increases Space Complexity.

Extend — Adding Input Size as a parent node

The map is extended by placing Input Size as a parent influencing Recursion Depth. This reflects that depth is not arbitrary — it is typically a function of the input size n. For example, a recursive factorial or linear recursion on n elements produces depth O(n), while a balanced divide-and-conquer recursion (like binary search) produces depth O(log n). Input Size therefore acts as the root driver that determines how deep the recursion tree will grow.

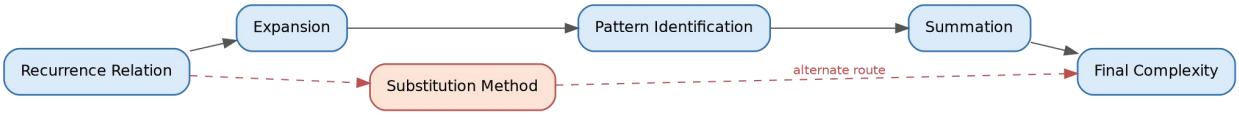
Justify — Why deep recursion increases stack usage

Since every recursive call must remain on the stack until it returns (its result may be needed to complete the parent call), the stack cannot discard a frame until that branch of recursion resolves. If the recursion depth is d, at least d frames exist simultaneously in the worst case just before the base case is reached. Because each frame has a fixed minimum size, total stack usage grows linearly with d — so deeper recursion directly and unavoidably increases memory consumption, independent of how much work each individual call does.

Interpret — How this map helps optimize recursive solutions

The map makes explicit the causal chain from problem size to memory cost: Input Size → Recursion Depth → Call Stack → Auxiliary Space → Space Complexity. This helps developers spot two levers for optimization: (1) reduce depth by restructuring the recursion (e.g., converting linear recursion into a divide-and-conquer form with logarithmic depth), or (2) eliminate the call stack altogether by converting recursion to iteration (using an explicit loop or an explicit stack data structure with controlled size). Seeing recursion depth as the direct driver of memory cost also helps in identifying when recursion is safe to use versus when it risks stack overflow on large inputs.

Q83. Solving Recurrence Relations Concept Map



blue = original map orange (dashed) = extension

Explain — How expansion helps identify patterns

Expansion means repeatedly substituting the recurrence into itself (unrolling it call by call) to reveal how the total cost accumulates across levels — for example, expanding $T(n) = T(n-1) + n^2$ into $T(n-2) + (n-1)^2 + n^2$, and so on. Writing out several levels this way exposes a repeating structure or arithmetic/geometric pattern (Pattern Identification) in the accumulated terms, which can then be expressed as a Summation formula, and finally solved in closed form to give the Final Complexity.

Extend — Linking the Substitution Method

The map is extended by adding a Substitution Method node connected from Recurrence Relation, with a dashed edge leading directly to Final Complexity. This represents an alternate solving route: rather than expanding step-by-step and summing, the substitution method guesses a closed-form solution (e.g., $T(n) = O(n \log n)$) and then proves it correct by induction — substituting the guess back into the recurrence to verify the inequality holds. Both routes (expansion-based summation and substitution) can independently arrive at the same Final Complexity.

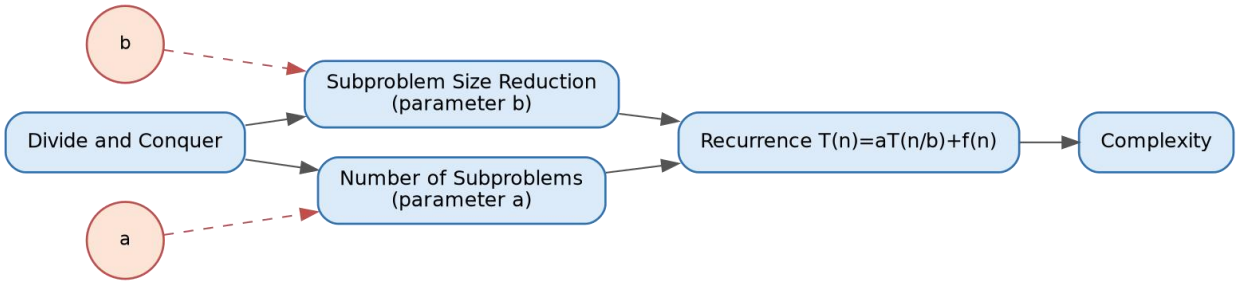
Justify — Why recognizing patterns matters for simplification

Without pattern recognition, one would need to manually expand a recurrence n times to compute its exact cost, which is impractical for large n. Recognizing that the expanded terms form a known series (arithmetic, geometric, harmonic, etc.) allows the use of a closed-form summation formula, collapsing an n-step manual expansion into a single algebraic expression. This turns an intractable case-by-case analysis into a fast, generalizable derivation — essential for reasoning about complexity at scale.

Interpret — How this map aids effective recurrence solving

The map lays out a clear decision path for tackling any recurrence: expand it, look for a pattern, sum that pattern, and simplify to a final bound — while also flagging substitution as a faster alternative when a plausible closed form can be guessed upfront. This structure helps students and practitioners avoid getting stuck on unfamiliar recurrences by giving them a repeatable method (expand → pattern → sum) as a fallback whenever a direct guess is not obvious, and encourages cross-checking one method's result against the other for confidence in the answer.

Q84. Divide and Conquer Recurrence Concept Map



blue = original map orange (dashed) = extension

Explain — How size reduction and subproblem count form a recurrence

A divide-and-conquer algorithm splits a problem of size n into a Number of Subproblems, each of a smaller size resulting from Subproblem Size Reduction. Together, these two structural choices define a recurrence of the standard form $T(n) = a \cdot T(n/b) + f(n)$, where the recursive term $a \cdot T(n/b)$ captures the recursive work (a subproblems, each of size n/b) and f(n) captures the non-recursive combine/merge cost at the current level.

Extend — Including parameters a and b

The map is extended with two small parameter nodes: a, feeding into Number of Subproblems, and b, feeding into Subproblem Size Reduction. This makes explicit that a and b are not abstract symbols but direct counts drawn from the algorithm's actual structure — a is literally how many recursive calls are made, and b is literally the factor by which the input shrinks at each call (e.g., merge sort has a = 2, b = 2).

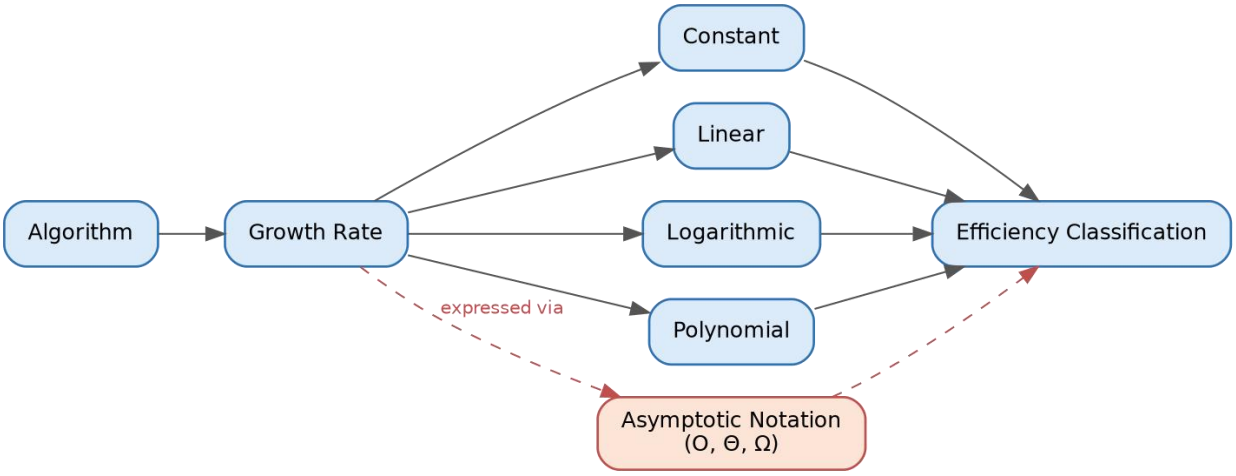
Justify — Their influence on the complexity outcome

The values of a and b determine the shape of the recursion tree and, through the Master Theorem, the final complexity: the ratio between a and b (expressed as $n^{\log_b a}$) sets a baseline growth rate that is then compared against f(n). A larger a (more subproblems) or a smaller b (less size reduction per level) both increase the total recursive work, pushing complexity higher; conversely, larger b or smaller a reduces recursive work. This is why algorithms with the same f(n) but different a, b values (e.g., $T(n)=2T(n/2)+n$ vs. $T(n)=4T(n/2)+n$) can end up with entirely different complexities ($\Theta(n \log n)$ vs. $\Theta(n^2)$).

Interpret — Connecting algorithm structure with analysis

This map shows that complexity analysis is not a separate, abstract exercise from algorithm design — it is a direct consequence of concrete structural decisions (how many pieces to split into, and how small each piece becomes). By tracing the path from Divide and Conquer through explicit parameters a and b to the Recurrence and then Complexity, a designer can predict — and deliberately steer — the performance of a new algorithm before implementing it, simply by choosing a favorable balance between the number of subproblems and how much each one shrinks.

Q85. Growth Rate and Efficiency Classification Concept Map



blue = original map orange (dashed) = extension

Explain — How algorithms are classified by growth rate

An algorithm's Growth Rate describes how its resource usage (typically time) increases as input size n grows. Algorithms are grouped into broad classes based on this growth behavior: Constant (O(1), unaffected by n), Logarithmic (O(log n), grows very slowly, typical of halving-based algorithms like binary search), Linear (O(n), grows proportionally with n), and Polynomial (O(n^k) for k > 1, grows faster, typical of nested loops). Each class maps to an Efficiency Classification that indicates how well the algorithm will scale to large inputs.

Extend — Linking Asymptotic Notation

The map is extended by adding an Asymptotic Notation node (O, Θ, Ω) connected from Growth Rate and leading into Efficiency Classification. This reflects that growth rates are not just informally described ("linear", "logarithmic") but are formally expressed using asymptotic bounds — Big-O for upper bounds, Big-Omega for lower bounds, and Big-Theta for tight bounds — which is the standard notation used to rigorously classify and compare algorithms.

Justify — Why lower growth rates are preferred

Lower growth rates mean an algorithm's resource consumption increases more slowly as input size scales up. For large or unpredictable input sizes, an O(n) algorithm will vastly outperform an O(n²) algorithm once n is sufficiently large — even if the O(n²) algorithm has a smaller constant factor and appears faster on tiny inputs. Choosing lower growth-rate algorithms is therefore essential for scalability, ensuring systems remain responsive and resource-efficient as data volume grows in real-world use.

Interpret — How this map supports selecting scalable algorithms

The map connects abstract growth-rate categories to the formal notation used to prove them, and ultimately to a practical efficiency classification a designer can act on. This gives a clear decision pipeline: analyze an algorithm's operations, express its growth using asymptotic notation, classify it into a growth-rate category, and use that classification to judge whether the algorithm is suitable for the expected input scale. It reinforces that algorithm selection should be driven by how performance changes with growing data, not just by how an algorithm performs on the small test cases typically used during development.